

Create Authentication Service that Returns JWT

1. Add Maven Dependency (pom.xml)

```
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt</artifactId>
  <version>0.9.0</version>
</dependency>
```

2. AuthenticationController.java

```
package com.cognizant.springlearn.controller;

import java.util.Base64;
import java.util.Date;
import java.util.HashMap;
import java.util.Map;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestHeader;
import org.springframework.web.bind.annotation.RestController;
import io.jsonwebtoken.JwtBuilder;
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.SignatureAlgorithm;

@RestController
public class AuthenticationController{

    private static final Logger
    LOGGER=LoggerFactory.getLogger(AuthenticationController.class);
```

```

@GetMapping("/authenticate")

public Map<String,String> authenticate(@RequestHeader("Authorization") String
authHeader){

    LOGGER.info("START");

    LOGGER.debug(authHeader);

    Map<String,String> map=new HashMap<>();

    String user=getUser(authHeader);
    String token=generateJwt(user);

    map.put("token",token);

    LOGGER.info("END");

    return map;
}

private String getUser(String authHeader){

    LOGGER.info("START");

    String encodedCredentials=authHeader.substring("Basic ".length());

    byte[] decodedBytes=Base64.getDecoder().decode(encodedCredentials);

    String credentials=new String(decodedBytes);

    String user=credentials.split(":")[0];

```

```

    LOGGER.debug(user);

    LOGGER.info("END");

    return user;
}

private String generateJwt(String user){

    JwtBuilder builder=Jwts.builder();

    builder.setSubject(user);
    builder.setIssuedAt(new Date());
    builder.setExpiration(new Date((new Date()).getTime()+1200000));
    builder.signWith(SignatureAlgorithm.HS256,"secretkey");

    return builder.compact();
}
}

```

3. SecurityConfig.java

```

@Override
protected void configure(HttpSecurity httpSecurity)throws Exception{

    httpSecurity.csrf().disable().httpBasic().and()
    .authorizeRequests()
    .antMatchers("/countries").hasRole("USER")
    .antMatchers("/authenticate").hasAnyRole("USER","ADMIN");
}

```

4. Request

```
curl -s -u user:pwd http://localhost:8090/authenticate
```

5. Output

```
{  
  
  "token": "eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJlc2VyliwiaWF0IjoxNTcwMzc5NDc0LCJleH  
AiOjE1NzAzODA2NzR9.t3LRvlCV-hwKfoqZYlaVQqEUiBloWcWn0ft3tgv0dL0"  
}
```

Result

The authentication service successfully reads the **Authorization** header, decodes the username, generates a **JWT token**, and returns it as a JSON response.